SimpleDB+ join experiments

Measured baseline on the current student database. These are local measurements from one machine, not results collected from multiple team members.

Setup and measurement

Database: student=9, enroll=6, section=5, course=6, dept=3 rows. Each query returned six rows. Platform: macOS 15.7, aarch64, 8 reported processors; OpenJDK 17.0.14. Database blocks: 400 bytes; buffer pool: 8 blocks. JVM: -Xms256m -Xmx512m.

For each of 20 configurations, three separate JVM runs each performed 10 discarded warm-ups and 30 measured executions: 90 measurements per configuration, 1,800 measured executions total. Configurations were shuffled using seeds 20261018, 20261019 and 20261020. Each configuration used its own database copy and transaction; the source database was never opened by SimpleDB.

System.nanoTime measured Plan.open() through full result consumption and Scan.close(). This includes sorting, partitioning, temporary-table work, and a common row-serialization consumer. Parsing/planning, database copying/recovery, validation, printing and transaction rollback were excluded. These are warmed, buffered runs, not cold-disk latency or pure inner-loop CPU measurements.

Experiment 1: two-table join

```
select sid, sname, eid, grade
from student, enroll
where sid = studentid
```

Every algorithm used the same orientation: student as the left/outer input and enroll as the right input. Each physical plan ends in the same projection. The method restriction is enforced, with no fallback to a different join.

Algorithm	Median (ms)	P95 (ms)
Tuple nested loop	0.065	0.292
Index nested loop	0.141	0.432
Sort-merge	1.172	2.544
Partitioned hash	1.984	4.171
Multi-buffer product + filter	1.383	4.550

Plans: nested loop tests sid=studentid while scanning; index join probes the existing enroll.studentid hash index for each student; sort-merge sorts both join keys; hash partitions and probes; multi-buffer materializes inputs, forms chunked products, then filters equality. Duplicate enrollments are preserved.

Experiment 2: four-table join

```
select sid, sname, eid, title, prof, grade
from student, enroll, section, course
where sid = studentid and sectionid = sectid
  and courseid = cid
```

Notation: S=student, E=enroll, T=section, C=course. All plans are left-deep and apply the same three equalities as soon as the relevant tables are joined, followed by the same projection.

Three plans selected by restricting the optimizer

Allowed method	Actual join order	Median ms	P95 ms
Tuple nested loop	(((T + E) + C) + S)	0.131	0.582
Partitioned hash	(((T + E) + S) + C)	4.147	6.974
Sort-merge	(((T + E) + S) + C)	5.284	24.728

These three configurations retain heuristic table ordering and allow only the named method. Their physical trees were checked to contain exactly three joins of that method. The different estimated output cardinalities also affect the heuristic's next-table choice.

Controlled comparison: three join orders x four algorithms

P1 = (((S + E) + T) + C); P2 = (((C + T) + E) + S); P3 = (((T + E) + S) + C). Here table order is fixed and only the allowed join method changes. This extra comparison separates method effects from optimizer-chosen order.

Order	Nested loop	Hash	Sort-merge	Multi-buffer
P1	0.172	3.860	4.618	4.173
P2	0.131	3.576	5.994	3.740
P3	0.142	6.629	6.153	4.020

Entries are median milliseconds over 90 executions. P95 values and each JVM's median are retained in summary.json; every raw duration is in samples.csv.

Index-only four-table plans are not applicable with the existing indexes: enroll.studentid is indexed, but section.sectid and course.cid are not. No extra indexes were created just to make that comparison work. Experiment 1 still measures the available index join directly.

All three orders produce small intermediate results: P1 has 6, 6, 6 joined rows; P2 has 5, 6, 6; P3 has 6, 6, 6. Record widths and which table is rescanned also differ. Detailed operator trees, join keys, restrictions and estimates are preserved in each fork's plans.txt.

Discussion and interpretation

Two-table result. Tuple nested loop was fastest by pooled median (0.065 ms), followed by the existing index join (0.141 ms). With only 9 x 6 candidate pairs, repeatedly scanning a tiny buffered table is inexpensive. Index setup/probes add overhead, while partitioning, sorting and materialization create temporary files. The file manager opens files in Java's synchronous 'rws' mode, making temporary writes relevant even for tiny relations.

Four-table result. The restricted nested-loop optimizer plan took 0.131 ms versus 4.147 ms for hash and 5.284 ms for merge. This does not demonstrate that nested loop scales better: it reflects a dataset where the entire useful input is only a few blocks. Hash and merge pay setup costs at each of the three join stages.

Join order. P2 had the lowest pooled median for fixed-order nested loop, hash and multi-buffer; P1 had the lowest for merge in this run. P2's first intermediate has five rows rather than six, but the small differences in size alone cannot establish causality. Sort order, intermediate width, temporary writes, JVM compilation and system load can all affect the observed timings.

Variability. The same physical plan is sometimes represented both by an optimizer-selected configuration and a fixed-order configuration. Their timings differ because they ran in separate blocks at different points in the JVM and system's execution. For example, the hash optimizer plan has the same T-E-S-C order as P3, yet their pooled medians differ. Therefore small rankings are descriptive, not statistically established. We retain outliers, report P95, and do not treat 90 within-process measurements as 90 independent JVM replications.

Planner limitations. EXPLAIN's rows and blocks are estimates, not measured counts or I/O. The existing distinct-value guesses and repeated selectivity reductions can even estimate zero rows for this nonempty join. This experiment preserves those estimates rather than changing the optimizer while measuring it. Actual result multisets were independently validated on every execution.

Reproduce and extend

```
javac -d /tmp/simpledb-benchmark-classes $(find src -name '*.java')
java -Xms256m -Xmx512m -cp /tmp/simpledb-benchmark-classes \
  simpledb.test.JoinExperiments studentdb output/joins/new-run \
  10 30 20261018
```

In Eclipse, run JoinExperiments as a Java application with the same five program arguments; use the project root as the working directory. Supply a new output directory each time. The runner prints median/min/P95 and saves raw samples, plans and environment information. Its explicit method restrictions reject unavailable joins instead of silently substituting another method.

For a team submission, add separately labelled teammate/machine measurements if required; none have been invented here. A useful follow-up is a larger generated student database and repeated trials under controlled cache conditions. These results support the implementation comparison on the current studentdb only, not a general ranking of database join algorithms.